
CS4100 Final Project

Release 0.1.0

Christian Garcia, Tommaso Maga, Yu-Chun Ou, Peter SantaLucia

Apr 06, 2026

CONTENTS:

1	cs4100-pcr-gradcam-prediction	1
1.1	License	1
1.2	Purpose	1
1.3	Existing Approaches	1
1.4	Datasets & Methodology	1
1.5	Works Cited	2
2	Modules	3
2.1	train	3
2.2	dataset	9
2.3	hirescam	10
2.4	visualize	11
2.5	pay_attn.model_outputs	11
	Python Module Index	13
	Index	15

CS4100-PCR-GRADCAM-PREDICTION

License CC BY-NC 4.0

Contributors: Christian Garcia, Tommaso Maga, Yu-Chun Ou, Peter SantaLucia

1.1 License

This project is licensed under [CC BY-NC 4.0](#) to comply with the licensing terms of the BreastDCEDL dataset, which itself adopts CC BY-NC 4.0 as the most restrictive license among its three source datasets from TCIA. Specifically, while I-SPY 1 (CC BY 3.0) and I-SPY 2 (CC BY 4.0) permit commercial use, the Duke Breast Cancer MRI dataset (CC BY-NC 4.0) does not. As a derivative work integrating all three sources, this project inherits that restriction. Academic use, research, and adaptation with attribution are welcome.

1.2 Purpose

The group has decided to predict patient breast cancer outcomes using MRI scans and patient metadata. An accurate measure of these outcomes is given by the pathological complete response (PCR) metric for triple-negative breast cancer (TNBC). That is, the complete remission of an invasive cancer that is found in a tissue sample. For TNBC Patients with PCR, 90% of patients experience event-free survival (EFS) over a period of three years, while only 67% experienced EFS over the same period (Toss, et al.). By predicting PCR for some patients, we are able to accurately predict patient outcomes over a period of three years. By having an accurate prediction of a patient's PCR, doctors can preemptively consider other treatment plans/evaluate the need for surgery for some patient. Given the aggressiveness of TNBC, it's widely studied, leading to many high-quality multiparametric MRI datasets available for academic use. With these datasets, we are able to train some model on 3D volumetric data that will help us predict long-term patient outcomes.

1.3 Existing Approaches

- [Nature Scientific Reports \(2025\)](#): Trained machine learning models to predict survival rates for patients undergoing breast cancer treatments. (*Note: Our project specifically builds on this domain by focusing on deep learning for 3D volumetric data and adding interpretability*).

1.4 Datasets & Methodology

- **Dataset:** [BreastDCEDL](#) A deep-learning-ready, labeled dataset that combines dynamic contrast-enhanced (DCE) MRI scans from three major clinical trials.
- **Environment:** Google Colab.
- **Architecture:** 3D Convolutional Neural Network (CNN) trained on volumetric MRI scans.

- **Interpretability:** We will implement **HiResCAM** to visualize the model’s decision-making process. Because medical decisions require high transparency, HiResCAM will ensure the generated heatmaps are mathematically faithful to the model’s internal weights.
- **Stretch Goal:** If time allows, we will implement custom positive/negative filters on the HiResCAM outputs. This toggle will allow clinicians to isolate the specific tissue features acting as positive contributors (evidence *for* pCR) versus negative contributors (evidence *against* pCR).

1.5 Works Cited

Draelos, Rachel Lea, and Lawrence Carin. “Use HiResCAM Instead of Grad-CAM for Faithful Explanations of Convolutional Neural Networks.” *arXiv*, 17 Nov. 2020, arxiv.org/abs/2011.08891.

Fridman, Naomi, et al. “BreastDCEDL: A Deep Learning–Ready Breast DCE-MRI Dataset.” *Zenodo*, 9 June 2025, doi.org/10.5281/zenodo.15627233.

Toss, Angela, et al. “Predictive Factors for Relapse in Triple-Negative Breast Cancer Patients without Pathological Complete Response after Neoadjuvant Chemotherapy.” *Frontiers in Oncology*, vol. 12, 1 Dec. 2022, doi.org/10.3389/fonc.2022.1016295.

2.1 train

Train a CNN using dataset.py

```
class train.ConvBlock(num_input_channels, num_output_channels)
```

Bases: Module

forward(x)

Define the computation performed at every call.

Should be overridden by all subclasses.

 Note

Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the registered hooks while the latter silently ignores them.

```
class train.PcrCNN(learning_rate=0.0001, pos_weight=1.0)
```

Bases: LightningModule, Module

configure_optimizers()

Choose what optimizers and learning-rate schedulers to use in your optimization. Normally you'd need one. But in the case of GANs or similar you might have multiple. Optimization with multiple optimizers only works in the manual optimization mode.

Returns

Any of these 6 options.

- **Single optimizer.**
- **List or Tuple** of optimizers.
- **Two lists** - The first list has multiple optimizers, and the second has multiple LR schedulers (or multiple `lr_scheduler_config`).
- **Dictionary**, with an "optimizer" key, and (optionally) a "lr_scheduler" key whose value is a single LR scheduler or `lr_scheduler_config`.
- **None** - Fit will run without any optimizer.

The `lr_scheduler_config` is a dictionary which contains the scheduler and its associated configuration. The default configuration is shown below.

```

lr_scheduler_config = {
    # REQUIRED: The scheduler instance
    "scheduler": lr_scheduler,
    # The unit of the scheduler's step size, could also be 'step'.
    # 'epoch' updates the scheduler on epoch end whereas 'step'
    # updates it after a optimizer update.
    "interval": "epoch",
    # How many epochs/steps should pass between calls to
    # `scheduler.step()`. 1 corresponds to updating the learning
    # rate after every epoch/step.
    "frequency": 1,
    # Metric to monitor for schedulers like `ReduceLROnPlateau`
    "monitor": "val_loss",
    # If set to `True`, will enforce that the value specified 'monitor'
    # is available when the scheduler is updated, thus stopping
    # training if not found. If set to `False`, it will only produce a warning
    "strict": True,
    # If using the `LearningRateMonitor` callback to monitor the
    # learning rate progress, this keyword can be used to specify
    # a custom logged name
    "name": None,
}

```

When there are schedulers in which the `.step()` method is conditioned on a value, such as the `torch.optim.lr_scheduler.ReduceLROnPlateau` scheduler, Lightning requires that the `lr_scheduler_config` contains the keyword "monitor" set to the metric name that the scheduler should be conditioned on.

Metrics can be made available to monitor by simply logging it using `self.log('metric_to_track', metric_val)` in your `LightningModule`.

Note

Some things to know:

- Lightning calls `.backward()` and `.step()` automatically in case of automatic optimization.
- If a learning rate scheduler is specified in `configure_optimizers()` with key "interval" (default "epoch") in the scheduler configuration, Lightning will call the scheduler's `.step()` method automatically in case of automatic optimization.
- If you use 16-bit precision (`precision=16`), Lightning will automatically handle the optimizer.
- If you use `torch.optim.LBFGS`, Lightning handles the closure function automatically for you.
- If you use multiple optimizers, you will have to switch to 'manual optimization' mode and step them yourself.
- If you need to control how often the optimizer steps, override the `optimizer_step()` hook.

forward(x)

Same as `torch.nn.Module.forward()`.

Parameters

- ***args** – Whatever you decide to pass into the forward method.

- ****kwargs** – Keyword arguments are also possible.

Returns

Your model's output

`on_test_epoch_end()`

Called in the test loop at the very end of the epoch.

`on_train_batch_end(outputs, batch, batch_idx)`

Checks for NaN weights at the end of every training batch.

`on_train_epoch_end()`

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

`on_validation_epoch_end()`

Called in the validation loop at the very end of the epoch.

`shared_step(batch)`

`test_step(batch, batch_idx)`

Operates on a single batch of data from the test set. In this step you'd normally generate examples or calculate anything of interest such as accuracy.

Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch_idx** – The index of this batch.
- **dataloader_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

Returns

- `Tensor` - The loss tensor
- `dict` - A dictionary. Can include any keys, but must include the key `'loss'`.
- `None` - Skip to the next batch.

```

# if you have one test dataloader:
def test_step(self, batch, batch_idx): ...

# if you have multiple test dataloaders:
def test_step(self, batch, batch_idx, dataloader_idx=0): ...

```

Examples:

```

# CASE 1: A single test dataset
def test_step(self, batch, batch_idx):
    x, y = batch

    # implement your own
    out = self(x)
    loss = self.loss(out, y)

    # log 6 example images
    # or generated text... or whatever
    sample_imgs = x[:6]
    grid = torchvision.utils.make_grid(sample_imgs)
    self.logger.experiment.add_image('example_images', grid, 0)

    # calculate acc
    labels_hat = torch.argmax(out, dim=1)
    test_acc = torch.sum(y == labels_hat).item() / (len(y) * 1.0)

    # log the outputs!
    self.log_dict({'test_loss': loss, 'test_acc': test_acc})

```

If you pass in multiple test dataloaders, `test_step()` will have an additional argument. We recommend setting the default value of 0 so that you can quickly switch between single and multiple dataloaders.

```

# CASE 2: multiple test dataloaders
def test_step(self, batch, batch_idx, dataloader_idx=0):
    # dataloader_idx tells you which dataset this is.
    x, y = batch

    # implement your own
    out = self(x)

    if dataloader_idx == 0:
        loss = self.loss0(out, y)
    else:
        loss = self.loss1(out, y)

    # calculate acc
    labels_hat = torch.argmax(out, dim=1)
    acc = torch.sum(y == labels_hat).item() / (len(y) * 1.0)

    # log the outputs separately for each dataloader
    self.log_dict({f"test_loss_{dataloader_idx}": loss, f"test_acc_{dataloader_
    idx}": acc})

```

Note

If you don't need to test you don't need to implement this method.

Note

When the `test_step()` is called, the model has been put in eval mode and PyTorch gradients have been disabled. At the end of the test epoch, the model goes back to training mode and gradients are enabled.

training_step(batch, batch_idx)

Here you compute and return the training loss and some additional metrics for e.g. the progress bar or logger.

Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch_idx** – The index of this batch.
- **dataloader_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

Returns

- **Tensor** - The loss tensor
- **dict** - A dictionary which can include any keys, but must include the key 'loss' in the case of automatic optimization.
- **None** - In automatic optimization, this will skip to the next batch (but is not supported for multi-GPU, TPU, or DeepSpeed). For manual optimization, this has no special meaning, as returning the loss is not required.

In this step you'd normally do the forward pass and calculate the loss for a batch. You can also do fancier things like multiple forward passes or something model specific.

Example:

```
def training_step(self, batch, batch_idx):
    x, y, z = batch
    out = self.encoder(x)
    loss = self.loss(out, x)
    return loss
```

To use multiple optimizers, you can switch to 'manual optimization' and control their stepping:

```
def __init__(self):
    super().__init__()
    self.automatic_optimization = False

# Multiple optimizers (e.g.: GANs)
def training_step(self, batch, batch_idx):
    opt1, opt2 = self.optimizers()
```

(continues on next page)

(continued from previous page)

```

# do training_step with encoder
...
opt1.step()
# do training_step with decoder
...
opt2.step()

```

Note

When `accumulate_grad_batches > 1`, the loss returned here will be automatically normalized by `accumulate_grad_batches` internally.

validation_step(batch, batch_idx)

Operates on a single batch of data from the validation set. In this step you'd might generate examples or calculate anything of interest like accuracy.

Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch_idx** – The index of this batch.
- **dataloader_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

Returns

- `Tensor` - The loss tensor
- `dict` - A dictionary. Can include any keys, but must include the key `'loss'`.
- `None` - Skip to the next batch.

```

# if you have one val dataloader:
def validation_step(self, batch, batch_idx): ...

# if you have multiple val dataloaders:
def validation_step(self, batch, batch_idx, dataloader_idx=0): ...

```

Examples:

```

# CASE 1: A single validation dataset
def validation_step(self, batch, batch_idx):
    x, y = batch

    # implement your own
    out = self(x)
    loss = self.loss(out, y)

    # log 6 example images
    # or generated text... or whatever
    sample_imgs = x[:6]
    grid = torchvision.utils.make_grid(sample_imgs)
    self.logger.experiment.add_image('example_images', grid, 0)

```

(continues on next page)

(continued from previous page)

```

# calculate acc
labels_hat = torch.argmax(out, dim=1)
val_acc = torch.sum(y == labels_hat).item() / (len(y) * 1.0)

# log the outputs!
self.log_dict({'val_loss': loss, 'val_acc': val_acc})

```

If you pass in multiple val dataloaders, `validation_step()` will have an additional argument. We recommend setting the default value of 0 so that you can quickly switch between single and multiple dataloaders.

```

# CASE 2: multiple validation dataloaders
def validation_step(self, batch, batch_idx, dataloader_idx=0):
    # dataloader_idx tells you which dataset this is.
    x, y = batch

    # implement your own
    out = self(x)

    if dataloader_idx == 0:
        loss = self.loss0(out, y)
    else:
        loss = self.loss1(out, y)

    # calculate acc
    labels_hat = torch.argmax(out, dim=1)
    acc = torch.sum(y == labels_hat).item() / (len(y) * 1.0)

    # log the outputs separately for each dataloader
    self.log_dict({f"val_loss_{dataloader_idx}": loss, f"val_acc_{dataloader_
    idx}": acc})

```

Note

If you don't need to validate you don't need to implement this method.

Note

When the `validation_step()` is called, the model has been put in eval mode and PyTorch gradients have been disabled. At the end of validation, the model goes back to training mode and gradients are enabled.

```
train.main()
```

2.2 dataset

Download the min crop dataset from <https://zenodo.org/records/18114231>.

DO NOT COMMIT DATASET.

```
class dataset.BreastDCEDataset(csv_dir, data_dir, split=Split.TRAIN)
```

Bases: Dataset

```
class dataset.Split(*values)
```

Bases: IntEnum

TEST = 2

TRAIN = 0

VAL = 1

```
dataset.get_path(pid, data_dir)
```

2.3 hirescam

```
class hirescam.HiResCam(model, device, model_name, target_layer_name)
```

Bases: object

Inference layer adapted from <https://github.com/rachellea/hirescam>, Draelos & Carin, 2021 (arXiv:2011.08891)

Context: CNNs compute gradients $K \in \mathbb{R}^{m \times m}$

Methodology: HiResCAM uses gradients computed by the CNN to produce a heatmap $L_{HiResCAM}^c \in \mathbb{R}^{d \times h \times w}$ that highlights important regions in the input volume for a given class c .

See test.py for example usage.

hirescam(target_grads: ndarray, target_activs: ndarray) → ndarray

Compute the HiResCAM heatmap using the formula:

$$L_{HiResCAM}^c = \sum_k \frac{\partial Y^c}{\partial A^k} \odot A^k$$

Parameters

- **target_grads** – Gradients at the target layer, shape (batch, num_features, D', H', W')
- **target_activs** – Activations at the target layer, shape (batch, num_features, D', H', W')

Returns

np.ndarray: Heatmap of shape (batch, D, H, W). NOTE that heatmap renderer is left to the caller.

return_explanation(ctvol, chosen_label_index)

Compute the HiResCam heatmap for a single input volume.

Parameters

- **ctvol** – Input volume of the shape e.g (1, 3, 32, 256, 256). The caller is responsible for ensuring this is the case.
- **chosen_label_index** – Index of the class for which to compute the heatmap. For binary classification, this will always be 0.

Returns

Raw CAM volume of shape (1,D,H,W), where D, H, W are the spatial dimensions of the target layer's output..

The caller is responsible for upsampling and visualizing this heatmap as desired.

2.4 visualize

Visualize HiResCAM attention maps overlaid on MRI slices for a single patient.

Pipeline:

1. Load model + one test sample
2. Run HiResCAM (via hirescam.py) at the last conv layer of the encoder
3. Upsample the 3D heatmap to the input volume size
4. For each of the 3 time points, show MRI slices side-by-side with the heatmap overlay

Note: The model ends in two FC layers, so HiResCAM's faithfulness guarantee (paper Section 3.5) does not strictly apply, but it is still more faithful than Grad-CAM.

```
visualize.load_model(path)
```

```
visualize.main()
```

```
visualize.overlay_heatmap_on_slice(mri_slice: ndarray, heat_slice: ndarray, alpha=0.5)
```

Blend a grayscale MRI slice with a jet-colormap heatmap. Both inputs should be in [0, 1]. Returns an RGB array.

```
visualize.upsample_heatmap(heatmap: Tensor, target_size: tuple) → Tensor
```

ReLU + normalize to [0,1] + trilinear upsample to target_size (D, H, W). heatmap: shape (D', H', W')

2.5 pay_attn.model_outputs

```
class model_outputs.ModelOutputsPcrCNN(model: PcrCNN, target_layer_name: str)
```

Bases: object

Class for running a PcrCNN <model> and: (1) Extracting activations from intermediate target layers (2) Extracting gradients from intermediate target layers (3) Returning the final model output

```
get_gradients()
```

```
run_model(x)
```

<p> Runs the model on input x, while saving activations and gradients from the target layer. </p> :param x: Input x is expected for (batch, 3, 32, 256, 256). The caller is responsible for ensuring this is the case. :return:

```
save_gradient(grad)
```

```
model_outputs.return_model_outputs_class(model_name)
```


PYTHON MODULE INDEX

d

`dataset`, [9](#)

h

`hirescam`, [10](#)

m

`model_outputs`, [11](#)

t

`train`, [3](#)

v

`visualize`, [11](#)

B

BreastDCEDataset (class in dataset), 9

C

configure_optimizers() (train.PcrCNN method), 3

ConvBlock (class in train), 3

D

dataset
module, 9

F

forward() (train.ConvBlock method), 3

forward() (train.PcrCNN method), 4

G

get_gradients() (model_outputs.ModelOutputsPcrCNN
method), 11

get_path() (in module dataset), 10

H

hirescam
module, 10

HiResCam (class in hirescam), 10

hirescam() (hirescam.HiResCam method), 10

L

load_model() (in module visualize), 11

M

main() (in module train), 9

main() (in module visualize), 11

model_outputs
module, 11

ModelOutputsPcrCNN (class in model_outputs), 11

module
dataset, 9
hirescam, 10
model_outputs, 11
train, 3
visualize, 11

O

on_test_epoch_end() (train.PcrCNN method), 5

on_train_batch_end() (train.PcrCNN method), 5

on_train_epoch_end() (train.PcrCNN method), 5

on_validation_epoch_end() (train.PcrCNN method),
5

overlay_heatmap_on_slice() (in module visualize),
11

P

PcrCNN (class in train), 3

R

return_explanation() (hirescam.HiResCam method),
10

return_model_outputs_class() (in module
model_outputs), 11

run_model() (model_outputs.ModelOutputsPcrCNN
method), 11

S

save_gradient() (model_outputs.ModelOutputsPcrCNN
method), 11

shared_step() (train.PcrCNN method), 5

Split (class in dataset), 10

T

TEST (dataset.Split attribute), 10

test_step() (train.PcrCNN method), 5

train
module, 3

TRAIN (dataset.Split attribute), 10

training_step() (train.PcrCNN method), 7

U

upsample_heatmap() (in module visualize), 11

V

VAL (dataset.Split attribute), 10

validation_step() (train.PcrCNN method), 8

visualize
module, 11